HOMEWORK TEIL 1 · OPTION A · TECHNICAL TRACK

MCP Inventory Agent

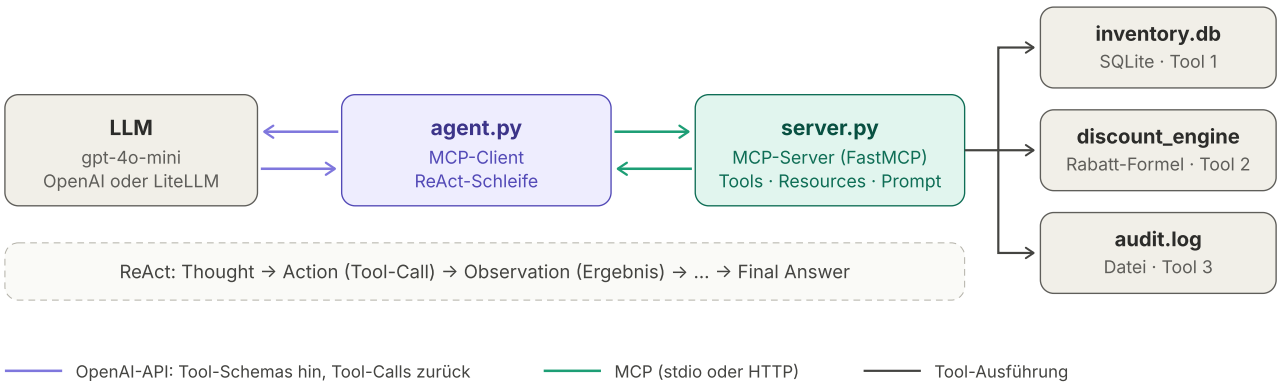
Eigener MCP-Server mit drei Tools – SQLite-Lagerabfrage, Formel-Engine für Staffelrabatte und Audit-Log in eine lokale Datei – und ein ReAct-Agent, der diese Tools zur Laufzeit über MCP entdeckt und nutzt. Das LLM ist wählbar: OpenAI direkt oder über einen LiteLLM-Proxy.

FACH
Industrial Computing
MCI Master · 3. Semester · Ch. Maurer

ERSTELLER
Andreas Rofner
Oktober 2026

REPOSITORY
ra0037I/mcp-inventory-agent
https://github.com/ra0037I/mcp-inventory-agent

Architektur



Installation

Python ≥ 3.11 · Git · OpenAI-API-Key

1 Repository klonen

```
git clone https://github.com/ra0037I/mcp-inventory-agent
cd mcp-inventory-agent
```

2 Umgebung anlegen und Pakete installieren

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Linux/macOS: `source .venv/bin/activate`

3 Konfiguration anlegen

```
copy .env.example .env
```

In der `.env` eintragen: `LLM_PROVIDER=openai` und `OPENAI_API_KEY=sk-…`

Starten

1 Demo starten

```
python agent.py --demo
```

Der Agent startet den MCP-Server selbst und durchläuft drei Szenarien: Lagerabfrage, Angebot über den Server-Prompt und Sonderfall mit Kategorie-Deckel und Fehlmenge.

2 Eigene Frage stellen

```
python agent.py "Which items to reorder?"
```

3 Ergebnisse prüfen

```
logs\agent_run_*.log  # ReAct-Spur
data\audit.log        # Audit-Log
```

4 Optional: über LiteLLM-Proxy

```
python start_litellm.py  # Terminal 1
python agent.py --demo   # Terminal 2
```

Dafür in der `.env`: `LLM_PROVIDER=litellm` und gleicher Wert für `LITELLM_KEY` / `LITELLM_MASTER_KEY`. Tests: `python -m pytest -q`